

$$(\text{Error at } x_i) \le \varepsilon \iff \begin{cases} b \ge -x_i w + (r_i - \varepsilon) \\ b \le -x_i w + (r_i + \varepsilon) \end{cases} \iff \begin{cases} b/w \ge (r_i - \varepsilon)/w - x_i \\ b/w \le (r_i + \varepsilon)/w - x_i \end{cases}$$

Fig. 9: Illustration of the proof of Lemma 1. Let $\mathcal{B}_- := \mathcal{B}(\delta_-)$, $\mathcal{B} := \mathcal{B}(0)$, and $\mathcal{B}_+ := \mathcal{B}(\delta_+)$. In the transformed $(1/w, b/w)$ space, moving the block along the key axis, i.e., from $\mathcal{B}_-$ to $\mathcal{B}$ to $\mathcal{B}_+$, corresponds to translating its feasible region along the b/w direction.

Appendix A
Proofs

Here, we provide the proofs we omitted in the main text.

A. Proof of Lemma 1

Proof of Lemma 1. The proof proceeds in three main steps: (1) defining the feasible region in a transformed parameter space, (2) deriving the translation of this region when the block is shifted, and (3) utilizing the Minkowski difference and convexity to establish the final error bound. Figure 9 illustrates the main idea.

1. Feasible Region and Parameter Transformation. We fix the error threshold $\varepsilon \ge 0$. For a given multiset $\mathcal{X} = \{x_1, x_2, \ldots, x_N\}$ with $x_1 \le x_2 \le \cdots \le x_N$, we define the feasible region $\mathcal{F}(\mathcal{X})$ in the (w, b) plane as:

$$\mathcal{F}(\mathcal{X}) := \{(w, b) \in \mathbb{R}^2 \mid \forall x_i \in \mathcal{X},\ |wx_i + b - r_i| \le \varepsilon\}, \quad (18)$$

where r_i is the rank of x_i in $\mathcal{X}$, i.e., $r_i := i$. Since each inequality $|wx_i + b - r_i| \le \varepsilon$ defines a convex region bounded by two parallel lines, their intersection $\mathcal{F}(\mathcal{X})$ is also a convex region.

Now, we apply the transformation $(u, v) = (1/w, b/w)$. Since both keys x_i and ranks r_i are non-decreasing, the optimal slope w is strictly positive. Therefore, the transformation $(u, v) = (1/w, b/w)$ is a bijection. In this transformed space, the inequality $|wx_i + b - r_i| \le \varepsilon$ is equivalent to:

$$(r_i - \varepsilon)u - v \le x_i \le (r_i + \varepsilon)u - v. \quad (19)$$

Let $\mathcal{F}'(\mathcal{X})$ denote the feasible region in the (u, v) space. Being defined by a set of linear inequalities, $\mathcal{F}'(\mathcal{X})$ is also a convex polygon.

2. Shift in the Transformed Space. The condition $\delta \in [x_{l-1} - x_l, x_{r+1} - x_r]$ ensures that the rank r_i for each key x_i remains constant. For the shifted block $\mathcal{B}(\delta) = \{x_i + \delta\}_{i=l}^{r}$, the feasibility condition becomes:

$$(r_i - \varepsilon)u - v \le x_i + \delta \le (r_i + \varepsilon)u - v \quad (20)$$

$$\iff (r_i - \varepsilon)u - (v + \delta) \le x_i \le (r_i + \varepsilon)u - (v + \delta). \quad (21)$$

This matches the condition for the original block $\mathcal{B}(0)$ evaluated at the point $(u, v + \delta)$. Therefore, the feasible region for the shifted block is simply the region for the original block shifted by $-\delta$ along the v-axis:

$$\mathcal{F}'(\mathcal{B}(\delta)) = \mathcal{F}'(\mathcal{B}(0)) + \boldsymbol{d}(\delta) \quad (22)$$

where $\boldsymbol{d}(\delta) := (0, -\delta)$.

3. Overlap and Convexity. Let $\mathcal{G} := \mathcal{F}'(\mathcal{A})$ and $\mathcal{H} := \mathcal{F}'(\mathcal{B}(0))$. Both $\mathcal{G}$ and $\mathcal{H}$ are convex sets. The combined dataset $\mathcal{A} \uplus \mathcal{B}(\delta)$ has an error of at most ε if and only if $\mathcal{G} \cap (\mathcal{H} + \boldsymbol{d}(\delta)) \ne \emptyset$.

Using the Minkowski difference $\mathcal{D} := \mathcal{G} - \mathcal{H} = \{\boldsymbol{g} - \boldsymbol{h} \mid \boldsymbol{g} \in \mathcal{G}, \boldsymbol{h} \in \mathcal{H}\}$, the intersection condition $\mathcal{G} \cap (\mathcal{H} + \boldsymbol{d}(\delta)) \ne \emptyset$ is equivalent to $\boldsymbol{d}(\delta) \in \mathcal{D}$. Because $\mathcal{G}$ and $\mathcal{H}$ are convex, their Minkowski difference $\mathcal{D}$ is also convex.

Now, let $\varepsilon = \max(E(\mathcal{A} \uplus \mathcal{B}(\delta_-)), E(\mathcal{A} \uplus \mathcal{B}(\delta_+)))$. By assumption, both extreme shifts achieve an error of at most ε, which implies:

$$\boldsymbol{d}(\delta_-) = (0, -\delta_-) \in \mathcal{D} \quad \text{and} \quad \boldsymbol{d}(\delta_+) = (0, -\delta_+) \in \mathcal{D}. \quad (23)$$

Since $\delta_- \le 0 \le \delta_+$, the shift $\delta = 0$ is a convex combination of δ_- and δ_+. Consequently, the zero vector $\boldsymbol{d}(0) = (0, 0)$ is a convex combination of $\boldsymbol{d}(\delta_-)$ and $\boldsymbol{d}(\delta_+)$. By the convexity of $\mathcal{D}$, it follows that $\boldsymbol{d}(0) \in \mathcal{D}$.

This implies $\mathcal{G} \cap \mathcal{H} \ne \emptyset$, meaning there exists a valid parameter pair (u, v) for the unshifted dataset. Thus, $E(\mathcal{A} \uplus \mathcal{B}(0)) \le \varepsilon$, which completes the proof. □

B. Proof of Theorem 1

Proof of Theorem 1. We prove the claim by contradiction. Assume that no optimal solution satisfies the stated property. That is, in every optimal solution, there exists at least one poison key that does not belong to any block adjacent to a legitimate key.

Among all optimal solutions, let $\mathcal{P}^*$ be one minimizing the number of *isolated poison blocks*, where an isolated poison block is a maximal consecutive-integer subset of poisons whose endpoints are not adjacent to any legitimate key. Formally, an *isolated poison block* is a set of poison keys $\{a, a+1, \ldots, b\} \subseteq \mathcal{P}^*$ such that $a - 1 \notin \mathcal{K} \cup \mathcal{P}^*$ and $b + 1 \notin \mathcal{K} \cup \mathcal{P}^*$.

By assumption, $\mathcal{P}^*$ contains at least one such block. Let $\mathcal{B}$ be one of them, and let $\mathcal{A} := (\mathcal{K} \cup \mathcal{P}^*) \setminus \mathcal{B}$. By Lemma 1, shifting $\mathcal{B}$ either left or right does not decrease the maximum-error value, while reducing the number of isolated poison blocks (see Figure 3). This contradicts the choice of $\mathcal{P}^*$, completing the proof. □

C. Proof of Theorem 2

Next, we provide the proof of Theorem 2.

Proof of Theorem 2. The proof proceeds in three main steps: (1) showing that any optimal solution $\mathcal{P}^*$ uses the full budget $|\mathcal{P}^*| = \lambda$, (2) showing that there exists an optimal $\mathcal{P}^*$ with $\mathrm{Supp}(\mathcal{P}^*) \subseteq \mathcal{K}$, and (3) showing that there exists an optimal $\mathcal{P}^*$ supported on a single integer $k \in \mathcal{K}$ (all poisons at the same value).

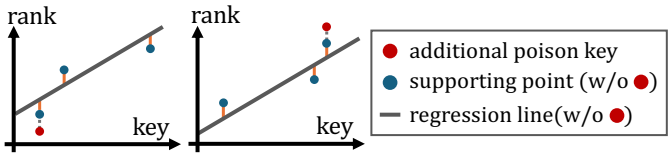


Fig. 10: Step 1 of the proof of Theorem 2: showing $|\mathcal{P}^*| = \lambda$. Regardless of how the three supporting points are arranged, we can always add a suitable poison to strictly increase the maximum error.

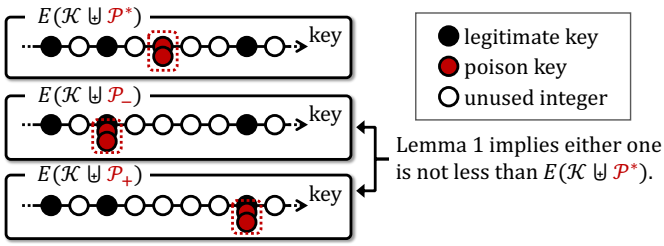


Fig. 11: Step 2 of the proof of Theorem 2: showing $\mathrm{Supp}(\mathcal{P}^*) \subseteq \mathcal{K}$. By shifting a block in $\mathrm{Supp}(\mathcal{P}^*) \setminus \mathcal{K}$ left or right, we can reduce $|\mathrm{Supp}(\mathcal{P}^*) \setminus \mathcal{K}|$ without decreasing the objective.

1. Full Budget at Optimality. It suffices to show that for any multiset $\mathcal{P}$ with $|\mathcal{P}| < \lambda$, there exists a key value p such that adding one copy of p to $\mathcal{P}$ strictly increases the maximum-error value $E(\mathcal{K} \uplus \mathcal{P})$. Such a p can be identified by examining the supporting points (see Figure 10).

Let $\mathcal{P}$ be a multiset with $|\mathcal{P}| < \lambda$, let $\varepsilon = E(\mathcal{K} \uplus \mathcal{P})$, and consider an optimal maximum-error regression line for $\mathcal{K} \uplus \mathcal{P}$. It is known that there exist three points $x_1, x_2, x_3 \in \mathcal{K} \uplus \mathcal{P}$ whose errors attain $\pm\varepsilon$ with alternating signs. If the signs are $-\varepsilon, +\varepsilon, -\varepsilon$ (left panel of Figure 10), choose $p = x_1$. If the signs are $+\varepsilon, -\varepsilon, +\varepsilon$ (right panel), choose $p = x_3$. In either case, no single line can regress the four points (i.e., x_1, x_2, x_3, and p) with maximum error ε, hence the optimum maximum error must strictly increase. Therefore, whenever $|\mathcal{P}| < \lambda$, we can strictly increase $E(\mathcal{K} \uplus \mathcal{P})$ by adding one poison, which implies $|\mathcal{P}^*| = \lambda$.

2. Support on Legitimate Keys. We prove the existence of an optimal solution supported only on legitimate keys by a contradiction argument similar to that of Theorem 1. Assume that every optimal poison set contains at least one value that is not a legitimate key. Among optimal solutions, choose $\mathcal{P}^*$ that minimizes $|\mathrm{Supp}(\mathcal{P}^*) \setminus \mathcal{K}|$. Then there exists a contiguous block of support values in $\mathrm{Supp}(\mathcal{P}^*) \setminus \mathcal{K}$. By shifting this block left or right (within the range where ranks do not change), we can apply the block-moving argument to obtain another optimal solution whose maximum error does not decrease, while $|\mathrm{Supp}(\mathcal{P}^*) \setminus \mathcal{K}|$ decreases (see Figure 11). This contradicts the minimality of $|\mathrm{Supp}(\mathcal{P}^*) \setminus \mathcal{K}|$. Hence, there exists an optimal solution with $\mathrm{Supp}(\mathcal{P}^*) \subseteq \mathcal{K}$.

3. Concentration on a Single Key. Finally, we show that any optimal poison set whose support is contained in $\mathcal{K}$ can be transformed into another optimal poison set supported on a single key value.

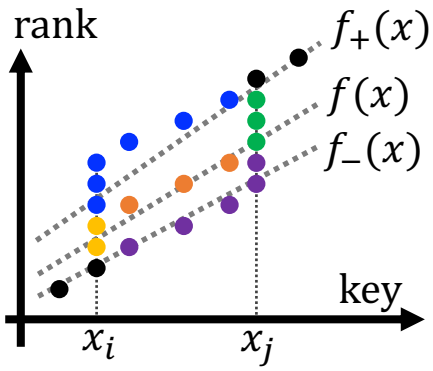


Fig. 12: Step 3 of the proof of Theorem 2. The figure illustrates the case where $d_i = 2$ and $d_j = 3$. Property 1 shows that the error of f with respect to the black, yellow, and green points is at most ε. Property 2 shows that the error for the orange points is at most ε.

For $\boldsymbol{d} \in \mathbb{Z}^n_{\geq 0}$, let $\mathcal{Q}_{\mathcal{K}}(\boldsymbol{d})$ denote the multiset that contains d_i copies of k_i for each i. Consider any $\boldsymbol{d}$ with $d_i > 0$ and $d_j > 0$ for some $i < j$. Define $\boldsymbol{d}_-$ by moving all copies at k_j to k_i, and define $\boldsymbol{d}_+$ by moving all copies at k_i to k_j, that is,

$$\boldsymbol{d} = [d_1, \ldots, d_{i-1}, \quad d_i, \ldots, \quad d_j, d_{j+1}, \ldots, d_n], \tag{24}$$

$$\boldsymbol{d}_- = [d_1, \ldots, d_{i-1}, \quad d_i + d_j, \ldots, \quad 0, d_{j+1}, \ldots, d_n], \tag{25}$$

$$\boldsymbol{d}_+ = [d_1, \ldots, d_{i-1}, \quad 0, \ldots, \quad d_i + d_j, d_{j+1}, \ldots, d_n]. \tag{26}$$

We claim that for any $\varepsilon \geq 0$,

$$E(\mathcal{K} \uplus \mathcal{Q}_{\mathcal{K}}(\boldsymbol{d}_-)) \leq \varepsilon \ \wedge \ E(\mathcal{K} \uplus \mathcal{Q}_{\mathcal{K}}(\boldsymbol{d}_+)) \leq \varepsilon \tag{27}$$

$$\implies E(\mathcal{K} \uplus \mathcal{Q}_{\mathcal{K}}(\boldsymbol{d})) \leq \varepsilon. \tag{28}$$

To see this, let $f_-(x)$ and $f_+(x)$ be regression lines achieving maximum error at most ε for $\mathcal{K} \uplus \mathcal{Q}_{\mathcal{K}}(\boldsymbol{d}_-)$ and $\mathcal{K} \uplus \mathcal{Q}_{\mathcal{K}}(\boldsymbol{d}_+)$, respectively. Define the convex combination

$$f(x) := \frac{d_i \cdot f_+(x) + d_j \cdot f_-(x)}{d_i + d_j}. \tag{29}$$

Then, we can show that $f(x)$ attains maximum error at most ε on $\mathcal{K} \uplus \mathcal{Q}_{\mathcal{K}}(\boldsymbol{d})$, which establishes Equation (28). To see this, it is sufficient to show the following two properties (each of which can be proved using elementary methods):

- **Property 1:** If a point (x, y) has an error of at most ε under both f_- and f_+, then its error under f is also at most ε.
- **Property 2:** If the points (x, y) and $(x, y + d_i + d_j)$ have errors of at most ε under f_- and f_+, respectively, then the error of the point $(x, y + d_i)$ under f is at most ε.

We provide Figure 12 to aid understanding. $\mathcal{K} \uplus \mathcal{Q}_{\mathcal{K}}(\boldsymbol{d}_-)$ corresponds to the black, purple, and green point sets; $\mathcal{K} \uplus \mathcal{Q}_{\mathcal{K}}(\boldsymbol{d}_+)$ corresponds to the black, yellow, and blue point sets; and

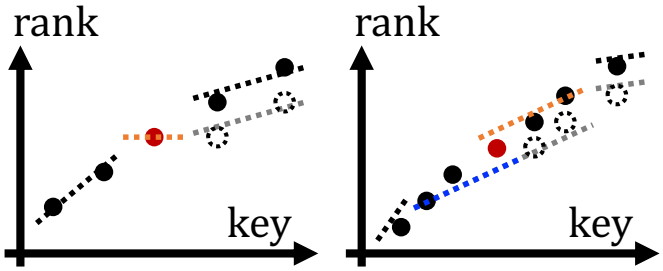


Fig. 13: Proof of Theorem 5. (1) Case 1 (left): when p lies between two adjacent segments, we add a new segment covering only p. (2) Case 2 (right): when p lies inside an existing segment, we split the segment containing p into two.

$\mathcal{K} \uplus \mathcal{Q}_{\mathcal{K}}(\boldsymbol{d})$ corresponds to the black, yellow, orange, and green point sets. By Property 1, it follows that the error of the black, yellow, and green point sets under f is at most ε. By Property 2, it follows that the error of the orange point set under f is at most ε. Thus, via Properties 1 and 2, it is shown that the error of $\mathcal{K} \uplus \mathcal{Q}_{\mathcal{K}}(\boldsymbol{d})$ under f is at most ε.

Now, setting $\varepsilon = \max\big(E(\mathcal{K} \uplus \mathcal{Q}_{\mathcal{K}}(\boldsymbol{d}_-)),\, E(\mathcal{K} \uplus \mathcal{Q}_{\mathcal{K}}(\boldsymbol{d}_+))\big)$ in Equation (28) yields immediately

$$E(\mathcal{K} \uplus \mathcal{Q}_{\mathcal{K}}(\boldsymbol{d})) \le \max\big(E(\mathcal{K} \uplus \mathcal{Q}_{\mathcal{K}}(\boldsymbol{d}_-)),\, E(\mathcal{K} \uplus \mathcal{Q}_{\mathcal{K}}(\boldsymbol{d}_+))\big). \tag{30}$$

Therefore, whenever poisons are distributed across multiple key values, we can merge their mass to one endpoint without decreasing the maximum error. By repeatedly applying this merging operation, we obtain an optimal solution whose poisons are supported on a single integer $k \in \mathcal{K}$. □

D. Proof of Theorem 5

Proof of Theorem 5. We first prove the case where one poison is added to $\mathcal{K}$; that is, we show that adding a single point p to $\mathcal{K}$ satisfies $m_{\mathrm{opt}}(\mathcal{K} \cup \{p\}, \varepsilon) \le m_{\mathrm{opt}}(\mathcal{K}, \varepsilon) + 1$. Now, let $\mathcal{S} = \big((s_1, e_1, f_1), (s_2, e_2, f_2), \ldots, (s_m, e_m, f_m)\big)$ be a PLA for $\mathcal{K}$ achieving $m \coloneqq m_{\mathrm{opt}}(\mathcal{K}, \varepsilon)$. The proof proceeds by considering the following two cases based on the position of p: **Case 1:** p lies between two adjacent segments in $\mathcal{S}$, and **Case 2:** p lies inside an existing segment in $\mathcal{S}$.

Case 1: More precisely, we consider the case where there exists some $j \in [m-1]$ such that $x_{e_j} < x_p < x_{s_{j+1}}$. In this case, we can construct a PLA for $\mathcal{K} \cup \{p\}$ with $m+1$ segments as follows (refer to the left side of Figure 13): We retain the first j segments (those preceding p) as they are, increase the function values of the $(j+1)$-th through m-th segments by one, and insert a new segment dedicated to covering p. It is clear that the approximation error remains at most ε for all keys in $\mathcal{K}$ and the poisoned point p.

Case 2: More precisely, we consider the case where there exists some $j \in [m]$ such that $x_{s_j} < x_p < x_{e_j}$. In this case, a PLA with $m+1$ segments can be constructed as follows (refer to the right side of Figure 13): We keep the first $j-1$ segments unchanged and increase the function values of the $(j+1)$-th through m-th segments by one. We then split the j-th segment into two parts at x_p, and increase the function value of the second part by one. Under the assumption that $\varepsilon \ge 1/2$, the point p can always be covered by at least one of these two segments within an error of ε. Thus, the approximation error for all keys and the poisoned point is guaranteed to be at most ε.

From **Case 1** and **Case 2**, it follows that $m_{\mathrm{opt}}(\mathcal{K} \cup \{p\}, \varepsilon) \le m_{\mathrm{opt}}(\mathcal{K}, \varepsilon) + 1$. We now complete the proof for the general case using induction on $|\mathcal{P}|$. Assume that $m_{\mathrm{opt}}(\mathcal{K} \cup \mathcal{P}, \varepsilon) \le m_{\mathrm{opt}}(\mathcal{K}, \varepsilon) + k$ holds for $|\mathcal{P}| = k$. For $|\mathcal{P}| = k+1$, let $\mathcal{P} = \mathcal{P}' \cup \{p\}$ where $|\mathcal{P}'| = k$. Then, we have:

$$\begin{aligned} m_{\mathrm{opt}}(\mathcal{K} \cup \mathcal{P}, \varepsilon) &= m_{\mathrm{opt}}((\mathcal{K} \cup \mathcal{P}') \cup \{p\}, \varepsilon) && (31)\\ &\le m_{\mathrm{opt}}(\mathcal{K} \cup \mathcal{P}', \varepsilon) + 1 && (32)\\ &\le m_{\mathrm{opt}}(\mathcal{K}, \varepsilon) + k + 1, && (33) \end{aligned}$$

where the first inequality follows from the result for a single-point insertion, and the second inequality follows from the inductive hypothesis. This concludes the proof for any $\mathcal{P}$. □

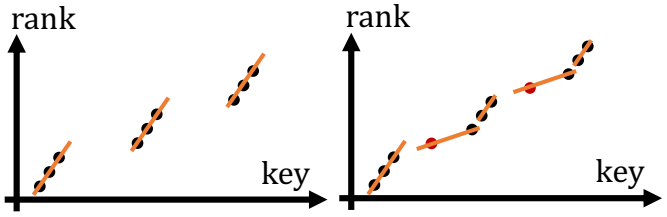


Fig. 14: An example of the tight upper bound (Theorem 6), where $\lambda = 2$ and $\varepsilon = 0$. Before poisoning (left), $m_{\mathrm{opt}}(\mathcal{K}, \varepsilon) = 3$ and after poisoning (right), $m_{\mathrm{opt}}(\mathcal{K} \cup \mathcal{P}, \varepsilon) = 5$.

E. Proof of Theorem 6

Proof of Theorem 6. The proof is constructive. Let $b \coloneqq 4\varepsilon + 3$ and $B \coloneqq (2\varepsilon+1)(2\varepsilon+2)+2$. We define a *block* as a set of b consecutive integers. We construct $\mathcal{K}$ by placing these blocks with a gap of $2B$ between each pair of adjacent blocks. The poisoning set $\mathcal{P}$ is then placed at the center of these gaps (that is, $|\mathcal{P}| = \lambda$). Precisely, we have

$$\mathcal{K} = \{2iB + j \mid i \in \{0, 1, \ldots, \lambda\}, j \in \{0, 1, \ldots, b-1\}\}, \tag{34}$$

$$\mathcal{P} = \{(2i+1)B + (2\varepsilon+1) \mid i \in \{0, 1, \ldots, \lambda-1\}\}. \tag{35}$$

In this configuration, we have $m_{\mathrm{opt}}(\mathcal{K}, \varepsilon) = \lambda + 1$. This is because an optimal PLA covers each block with exactly one segment, and no single segment can cover more than one block. This property is guaranteed by the fact that b and B are chosen to be sufficiently large.

Furthermore, it can be shown that $m_{\mathrm{opt}}(\mathcal{K} \cup \mathcal{P}, \varepsilon) = 2\lambda + 1$. This can be verified by greedily determining the segments from left to right:

- The 1-st segment covers the first block of $\mathcal{K}$.
- For $i \in \{1, 2, \ldots, \lambda\}$, the $2i$-th segment covers the i-th poisoned point in $\mathcal{P}$ and the first $2\varepsilon + 1$ elements of the $(i+1)$-th block.

Algorithm 2 DI-CONSECUTIVE

Input: Legitimate key set $\mathcal{K}$, poison budget λ
Output: Poison set $\mathcal{P}^*$

```
Precompute slope intervals 𝓘 and build sparse tables
C* ← C_ε(𝒦, ∅), 𝒫* ← ∅
for each candidate consecutive poison set 𝒫 with |𝒫| = λ do
    C_𝒫 ← C_ε(𝒦, 𝒫) ▷ time 𝒪(|𝓘| log c) via sparse tables
    if C_𝒫 < C* then
        C* ← C_𝒫, 𝒫* ← 𝒫
return 𝒫*
```

- For $i \in \{1, 2, \ldots, \lambda\}$, the $(2i+1)$-th segment covers the remaining $2\varepsilon + 2$ elements of the $(i+1)$-th block.

Thus, in this specific case, the equality in Equation (17) is satisfied, completing the proof. □

APPENDIX B
DETAILS ON DI-CONSECUTIVE

In this section, we provide the technical details omitted from the main text regarding DI-CONSECUTIVE (Discrete-Intercept Consecutive method) proposed in Section IV-C. The overall procedure is summarized in Algorithm 2.

Main Idea. The core objective of DI-CONSECUTIVE is to optimize the evaluation of each candidate of consecutive poisons, i.e., to compute the number of covered legitimate keys. We reduce the computational complexity from the conventional $\mathcal{O}(c)$, where c is the number of covered legitimate keys without poisoning, to $\mathcal{O}(|\mathcal{I}| \log c)$, where $|\mathcal{I}|$ is the number of candidate intercepts. In typical settings where $c \geq 10^4$ and $|\mathcal{I}| \approx 41$, this improvement is substantial.

Figure 15 illustrates the intuition behind our approach. The figure shows the evaluation of a candidate consecutive poisons starting from k_3+1. DI-CONSECUTIVE independently computes the range of slopes that can cover: (1) legitimate keys preceding the poisons, (2) legitimate keys following the poisons, and (3) the consecutive integer sequence containing the poisons. The candidate poisons and keys are coverable if and only if these three slope ranges have a non-empty intersection.

A. Precomputation

For each candidate intercept $b \in \mathcal{I}$, we perform a precomputation step with a time complexity of $\mathcal{O}(c \log c)$. Consequently, the total precomputation complexity is $\mathcal{O}(|\mathcal{I}| c \log c)$. Below, we describe the precomputation step for a particular b.

First, we compute $\underline{u}_i, \overline{u}_i, \underline{v}_i, \overline{v}_i$ for $i \in \{2, 3, \ldots, c\}$ as follows:

$$\underline{u}_i := \frac{(i-\varepsilon)-(1+b)}{k_i - k_1}, \quad \overline{u}_i := \frac{(i+\varepsilon)-(1+b)}{k_i - k_1}, \tag{36}$$

$$\underline{v}_i := \frac{(i+\lambda-\varepsilon)-(1+b)}{k_i - k_1}, \quad \overline{v}_i := \frac{(i+\lambda+\varepsilon)-(1+b)}{k_i - k_1}. \tag{37}$$

The interval $[\underline{u}_i, \overline{u}_i]$ represents the slope range required to cover k_i when it precedes the poisons. For example, in Figure 15, the slope of the line connecting $(k_1, 1+b)$ and $(k_3, 3+\varepsilon)$ corresponds to $\overline{u}_3$. Similarly, $[\underline{v}_i, \overline{v}_i]$ represents the slope range required to cover the consecutive integer sequence containing the poisons when k_i follows the poisons. For instance, in Figure 15, the slope of the line connecting $(k_1, 1+b)$ and $(k_6, 6+\lambda+\varepsilon)$ corresponds to $\overline{v}_6$. These values can be computed in $\mathcal{O}(c)$ time.

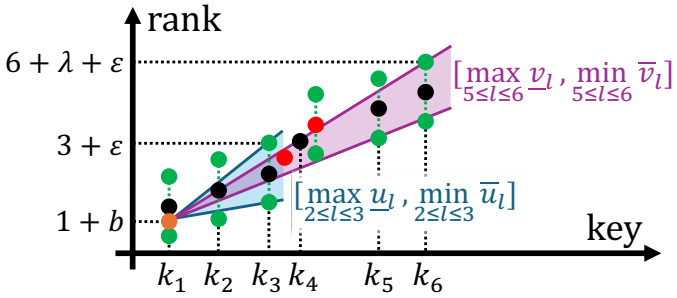


Fig. 15: An example of DI-CONSECUTIVE evaluating a candidate poison sequence (starting from k_3+1 with $\lambda = 2$). For a specific intercept b, slope ranges for pre-poison (k_2, k_3) and post-poison (k_5, k_6) keys are retrieved in $\mathcal{O}(1)$ via sparse tables. The slope range for the poisons is also computed in $\mathcal{O}(1)$ by finding the range covering k_3+1 and the largest poison.

Furthermore, we construct sparse tables over these four arrays to support $\mathcal{O}(1)$ Range Maximum Queries for $\underline{\boldsymbol{u}}$ and $\underline{\boldsymbol{v}}$, and $\mathcal{O}(1)$ Range Minimum Queries for $\overline{\boldsymbol{u}}$ and $\overline{\boldsymbol{v}}$. This construction requires $\mathcal{O}(c \log c)$ time.

B. Evaluation

The evaluation phase computes, for a given candidate consecutive poison set $\mathcal{P}$, the maximum number of legitimate keys that can be covered.

To achieve a time complexity of $\mathcal{O}(|\mathcal{I}| \log c)$ for each candidate consecutive poison set, we design the evaluation so that, for each intercept $b \in \mathcal{I}$, the number of coverable legitimate keys can be computed in $\mathcal{O}(\log c)$ time. This $\mathcal{O}(\log c)$ computation is realized via binary search combined with an $\mathcal{O}(1)$ feasibility check, which determines whether, for the given poison set, intercept, and keys, there exists a slope under which they are coverable.

Below, we first describe the **Maximal Coverage Search**, which computes in $\mathcal{O}(\log c)$ time the number of legitimate keys coverable for a fixed poison set and intercept. We then explain the **Coverability Verification**, which determines in $\mathcal{O}(1)$ time whether the given poison set, intercept, and keys are feasible.

Maximal Coverage Search. First, for a given candidate consecutive poison set $\mathcal{P}$, we find the smallest legitimate key greater than the largest poison in the set, and denote its index by j. For instance, in Figure 15, $j = 5$. A straightforward method would inspect the integers not occupied by legitimate keys in increasing order. In the worst case, this takes $\mathcal{O}(c+\lambda)$ time. To reduce this cost, we precompute the number of unused

integers between each pair of consecutive legitimate keys, as well as their prefix sums. Using this information, we can perform a binary search to determine where the largest poison falls relative to the gaps between legitimate keys, thereby identifying the first legitimate key after the poisons in $\mathcal{O}(\log c)$ time.

Next, for each intercept $b \in \mathcal{I}$, we perform a binary search to find the maximum $r \in \{j, \ldots, c\}$ such that $\{k_1, \ldots, k_r\} \cup \mathcal{P}$ are coverable. At each step, we invoke the **Coverability Verification** (described below) to determine whether $\{k_1, \ldots, k_r\} \cup \mathcal{P}$ are coverable. This yields the maximum feasible r for the given intercept in $\mathcal{O}(\log c)$ time.

After evaluating all $b \in \mathcal{I}$, we select the intercept that yields the largest r, which corresponds to the maximum number of covered legitimate keys for the given poison set.

Coverability Verification. The binary search requires verifying whether the set $\{k_1, \ldots, k_r\} \cup \mathcal{P}$ is coverable for a given intercept b. We determine the existence of a valid slope in $\mathcal{O}(1)$ time. Such a slope exists if and only if the following three ranges have a non-empty intersection: (1) the pre-poison range, (2) the post-poison range, and (3) the poison region range.

(1) The pre-poison range is the slope range covering legitimate keys preceding the poisons (i.e., from k_2 to k_i). This is computed as

$$\left[\max_{2\le l\le i} \underline{u}_l,\ \min_{2\le l\le i} \overline{u}_l\right]. \tag{38}$$

We can obtain this in $\mathcal{O}(1)$ time using a sparse table constructed during the precomputation step.

(2) The post-poison range is the slope range covering legitimate keys following the poisons (i.e., from k_j to k_r). This is computed as

$$\left[\max_{j\le l\le r} \underline{v}_l,\ \min_{j\le l\le r} \overline{v}_l\right]. \tag{39}$$

We can obtain this in $\mathcal{O}(1)$ time using the precomputed sparse table.

(3) The poison region range is the slope range covering the consecutive integers including the poisons (i.e., the consecutive integers from $k_i + 1$ to the largest poison). This slope range is obtained by intersecting the slope range that covers $k_i + 1$ with the slope range that covers the largest poison. This is because, by linearity, if the error at both endpoints is at most ε, then the error at every point between them is also guaranteed to be at most ε.

Appendix C
Details on the Instance-Dependent Upper Bound Algorithm

In this section, we provide the details omitted in the main text regarding the instance-dependent upper-bound algorithm proposed in Section V-D. Figure 16 illustrates the pipeline: partitioning into blocks, upper-bounding the number of segments in each block, and then allocating the total poison budget across blocks.

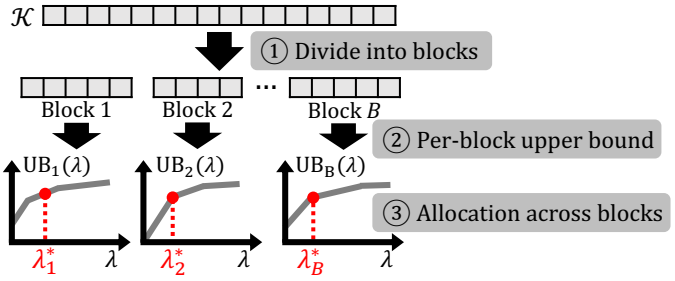


Fig. 16: Overview of the instance-dependent upper-bound algorithm: partition $\mathcal{K}$ into blocks with fixed slopes, compute per-block upper bounds, and allocate the poison budget across blocks to maximize their sum.

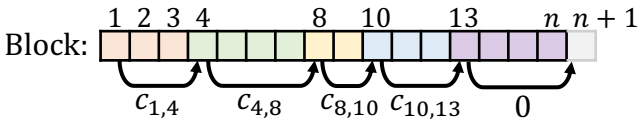


Fig. 17: Example of a path in the DAG for upper-bounding the number of segments in a block. A path represents a segmentation; its length gives the number of segments, and its total weight lower-bounds the required number of poisons.

A. Dividing into Blocks

We partition $\mathcal{K}$ into blocks using an $\alpha\varepsilon$-PLA with $\alpha \ge 1$ and fix the slope within each block to that of the corresponding PLA segment. This restricts the PLA construction by requiring each block to use a predetermined slope and by treating the blocks independently. We also strengthen the attacker by allowing duplicate poisons. Since these modifications weaken the index constructor and strengthen the attacker, the maximum number of segments in the resulting relaxed problem upper-bounds that in the original problem.

In our implementation, we evaluate the upper bound for $\alpha \in \{1.0, 1.2, 1.4, 1.6, 1.8, 2.0\}$ and take the minimum. Since each value of α yields a valid upper bound, taking their minimum preserves the upper-bound guarantee.

B. Per-Block Upper Bound

We next derive an upper bound on the number of segments in a single block. Let $\{k_1, k_2, \ldots, k_n\}$ be the keys in the block, and let w be its fixed slope.

First, we prove the following lemma concerning the minimum number of poisons required to separate two keys.

Lemma 2. *Let $\{k_1, k_2, \ldots, k_n\}$ be the keys in the block, and let w be its fixed slope. Define $A_l := l - wk_l$ for $l \in [n]$. Then, the attacker requires at least $\lceil c_{i,j} \rceil$ poisons to make the segment starting at k_i end before k_j, where*

$$c_{i,j} := \max\left(0, 2\varepsilon - \left(\max_{i\le l\le j} A_l - \min_{i\le l\le j} A_l\right)\right). \tag{40}$$

Proof of Lemma 2. Under a fixed slope w, let b be the optimal intercept for the set of keys $\{k_i, \ldots, k_j\}$. The prediction error

of the position for each key k_l ($i \le l \le j$) is expressed as $|(wk_l + b) - l| = |b - (l - wk_l)| = |b - A_l|$. Therefore, the optimal intercept that minimizes the maximum error in this segment is $b^* = \frac{1}{2}(\max_{i \le l \le j} A_l + \min_{i \le l \le j} A_l)$, and the initial maximum error E_{init} before poisoning is given by:

$$E_{\text{init}} = \frac{1}{2}\left(\max_{i \le l \le j} A_l - \min_{i \le l \le j} A_l\right) \tag{41}$$

Moreover, we can show that each duplicate poison can increase the maximum error by at most $1/2$. This holds because the linear regression can limit the increase in the maximum error to $1/2$ by increasing the intercept b by $1/2$ for each added poison.

Therefore, at least

$$\left\lceil \max\left(0, \frac{\varepsilon - E_{\text{init}}}{1/2}\right)\right\rceil = \lceil c_{i,j} \rceil \tag{42}$$

poisons are needed to make the error exceed ε. □

We now formulate the per-block problem as a path problem on a DAG. The DAG contains nodes $1, \ldots, n+1$, where node $n+1$ is a sentinel representing the end of the block. For each $1 \le i < j \le n$, we add an edge from i to j with weight $c_{i,j}$. We also add an edge of weight zero from each node $i \in [n]$ to the sentinel node $n+1$; that is, define $c_{i,n+1} = 0$ for all $i \in [n]$.

As illustrated in Figure 17, each path from node 1 to node $n+1$ represents a segmentation of the block. The starting node of each edge corresponds to the starting position of a segment, so the number of edges in the path equals the number of segments. By Lemma 2, the weight $c_{i,j}$ lower-bounds the number of poisons required to force the segment starting at k_i to end before k_j. The final edge has weight zero because the last segment naturally terminates at the end of the block. Consequently, the total weight of a path lower-bounds the number of poisons required to realize the corresponding segmentation.

Next, to reduce the computational complexity of finding shortest paths in the DAG, we prove the following lemma regarding the Monge property of $c_{i,j}$.

Lemma 3. *The cost function $c_{i,j}$ satisfies the Monge property, i.e., for any $1 \le i < i' < j < j' \le n$,*

$$c_{i,j} + c_{i',j'} \le c_{i,j'} + c_{i',j}. \tag{43}$$

Proof of Lemma 3. Let $D(i,j) := \max_{i \le l \le j} A_l - \min_{i \le l \le j} A_l$. For any $i < i' < j < j'$, consider the intervals $I_1 = [i,j]$ and $I_2 = [i',j']$. Their union is $I_1 \cup I_2 = [i,j']$ and their intersection is $I_1 \cap I_2 = [i',j]$. By the basic properties of the range over intervals, the sum of the ranges of the union and intersection is upper-bounded by the sum of the ranges of the original intervals:

$$D(i,j') + D(i',j) \le D(i,j) + D(i',j'). \tag{44}$$

Furthermore, the range is monotonically non-decreasing with respect to interval inclusion, meaning $D(i,j') \ge \max(D(i,j), D(i',j'))$. Since $c_{p,q} = \max(0,\, 2\varepsilon - D(p,q))$, these two properties together imply $c_{i,j} + c_{i',j'} \le c_{i,j'} + c_{i',j}$, concluding the proof. □

We finally derive an upper bound on the number of segments under a poison budget. For a path P from node 1 to node $n+1$, let $|P|$ denote its number of edges and let

$$C(P) \coloneqq \sum_{(i,j) \in P} c_{i,j} \tag{45}$$

denote its total weight. For any parameter $\nu > 0$, define

$$D(\nu) \coloneqq \min_{P} \left(C(P) - \nu |P|\right) \tag{46}$$

Equivalently, $D(\nu)$ is the shortest-path distance from node 1 to node $n+1$ after subtracting ν from every edge weight in the DAG.

For any path P realizable with at most λ poisons, we have $C(P) \le \lambda$. By the definition of $D(\nu)$,

$$D(\nu) \le C(P) - \nu|P| \le \lambda - \nu|P|. \tag{47}$$

Therefore, for any path P realizable with at most λ poisons,

$$|P| \le \frac{\lambda - D(\nu)}{\nu}. \tag{48}$$

Thus, Equation (48) gives an upper bound on the number of segments achievable within the block.

Subtracting the same constant ν from every edge preserves the Monge property. Hence, $D(\nu)$ can be computed in $\mathcal{O}(n)$ time using the LARSCH algorithm [20], [21]. In our experiments, we evaluate the bound for $\nu \in \mathcal{N} \coloneqq \{1, 2, 3, 4, 5, 10, 20, 40, 80, 160\}$ and take the minimum. For a block b and an allocated budget λ, we denote the resulting bound by $\mathrm{UB}_b(\lambda)$, defined as follows:

$$\mathrm{UB}_b(\lambda) \coloneqq \min_{\nu \in \mathcal{N}} \frac{\lambda - D_b(\nu)}{\nu}, \tag{49}$$

where $D_b(\nu)$ is the shortest-path distance from node 1 to node $n+1$ after subtracting ν from every edge weight in the DAG for block b.

C. Allocation Across Blocks

Suppose that $\mathcal{K}$ is divided into B blocks. Let λ_b be the number of poisons allocated to block b. Since the total poison budget is λ, the allocations satisfy

$$\sum_{b=1}^{B} \lambda_b \le \lambda. \tag{50}$$

Using the per-block bound in Equation (49), the global upper bound is obtained by solving

$$\max_{\substack{\lambda_1, \ldots, \lambda_B \in \mathbb{Z}_{\ge 0} \\ \sum_{b=1}^{B} \lambda_b \le \lambda}} \sum_{b=1}^{B} \mathrm{UB}_b(\lambda_b). \tag{51}$$

Each $\mathrm{UB}_b(\lambda_b)$ is the pointwise minimum of affine functions of λ_b and is therefore concave. Consequently, its marginal gain

$$\mathrm{UB}_b(\lambda_b + 1) - \mathrm{UB}_b(\lambda_b) \tag{52}$$

is non-increasing in λ_b. The allocation problem in Equation (51) can therefore be solved optimally by repeatedly assigning one poison to the block with the largest current marginal gain [23]. The sum of the resulting per-block bounds gives the instance-dependent upper bound for the selected block partition.

APPENDIX D
ADDITIONAL EXPERIMENTS

A. Additional Results on PLA Segment Maximization

Here, we present a comprehensive set of results on the post-poisoning m_{opt} and the upper bounds we derive, which were omitted from the main text. As attack methods, in addition to PGM-ATTACK, we also evaluate RANDOM and RANDOM-ADJACENT, the same baselines used in Section VI-B. As upper bounds, we evaluate the instance-agnostic upper bound (**Agnostic UB**; Section V-C) and the instance-dependent upper bound (**Instance UB**; Section V-D). We use $\varepsilon \in \{16, 32, 64, 128\}$. The numbers in parentheses indicate the factor relative to m_{opt} for the legitimate keys.

1) Complete Results on Poisoning and Upper Bounds: Table V shows the results. Across all rows of Table V, PGM-ATTACK attains a higher post-poisoning m_{opt} than both RANDOM and RANDOM-ADJACENT. On the YCSB dataset, PGM-ATTACK increases m_{opt} by $15.5\times$ with 1% poisoning and by up to $120\times$ with 10% poisoning. This pronounced effect stems from the fact that YCSB is nearly uniform, allowing a PLA with very few segments to be constructed before poisoning. By adding only a small number of poison keys, our attack drastically increases the required number of segments. Random poisoning also has a relatively large impact on YCSB: at a 10% poisoning rate, RANDOM and RANDOM-ADJACENT increase m_{opt} by $1.99\times$ and $2.42\times$, respectively. Nevertheless, the increase caused by PGM-ATTACK is substantially larger. The impact of our attack is also significant on datasets other than YCSB. At a 10% poisoning rate, it increases m_{opt} by factors ranging from $1.46\times$ to $24.5\times$, substantially exceeding the increases caused by the random baselines, which range from $1.00\times$ to $1.30\times$.

Regarding the upper bounds, Instance UB is consistently tighter than Agnostic UB and is at most $1.92\times$ the attained m_{opt}. This provides an instance-wise certificate that PGM-ATTACK achieves at least 52% of the optimum on all evaluated instances. It also shows that incorporating instance-specific information yields a tighter bound than the closed-form Agnostic UB. These results support the usefulness of our upper bounds for understanding the maximum achievable impact of poisoning attacks and, more broadly, for characterizing how much m_{opt} can increase under arbitrary insertions.

2) Robustness to Unknown Index Parameters: We further evaluate whether PGM-ATTACK remains effective when the attacker does not know the true PGM-index parameter. Specifically, we generate poison keys using an assumed value $\varepsilon_{\text{attack}}$ and construct the index using the actual value $\varepsilon_{\text{true}}$. Table VI reports the resulting m_{opt}.

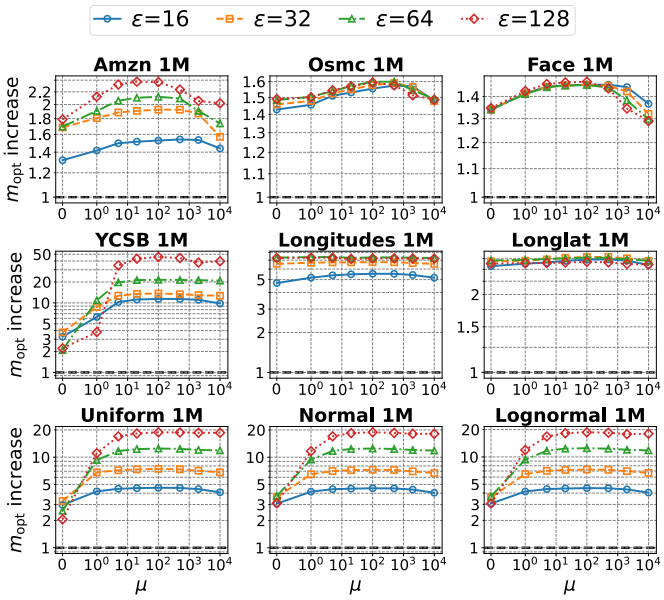


Fig. 18: Effect of μ on m_{opt} for $\lambda = 100{,}000$.

As expected, the attack is generally most effective when $\varepsilon_{\text{attack}} = \varepsilon_{\text{true}}$. Nevertheless, PGM-ATTACK remains effective even under parameter mismatch. For example, on Amzn with $\varepsilon_{\text{true}} = 128$, the attack increases m_{opt} by $2.50\times$ when the attacker knows the true parameter and sets $\varepsilon_{\text{attack}} = 128$. When the attacker instead assumes $\varepsilon_{\text{attack}} = 32$, the increase decreases to $1.36\times$. Even so, this remains larger than the increases caused by random and random-adjacent poisoning, which are $1.01\times$ and $1.17\times$, respectively (Table V). Similarly, on YCSB with $\varepsilon_{\text{true}} = 128$, mismatched attacks still increase m_{opt} by at least $22.7\times$, far exceeding the $1.99\times$ and $2.42\times$ increases caused by random and random-adjacent poisoning.

These results suggest that poison keys generated to target a PLA with one value of ε partially transfer to PLAs constructed with other values. Thus, exact knowledge of ε is not necessary to generate effective poison keys. In this sense, our white-box attack already remains effective in a *gray-box* setting where the attacker knows the data but not the value of ε. Our method therefore provides a useful foundation for developing more sophisticated attacks under gray-box and black-box settings.

3) Ablation Study on μ: To study the effect of μ, we report in Figure 18 the results averaged over five seeds on $n = 1$M keys. For $\varepsilon \in \{16, 32, 64, 128\}$, we plot the factor by which m_{opt} increases relative to legitimate keys. Very small or very large μ can reduce the amplification: small μ under-penalizes poison usage and exhausts budget unevenly, while large μ overreacts to budget imbalance and behaves unstably. Empirically, $\mu \approx 100$ works robustly; dynamic θ improves over fixing $\theta = \theta_0$ ($\mu = 0$) by up to $20\times$. These results show that dynamically updating θ yields higher measured m_{opt} than fixing $\theta = \theta_0$.

4) Poison Generation Time: Table VII reports the poison generation time for all combinations of ε and λ. Across

TABLE V: Results for m_{opt} after poisoning ($\lambda = 0.01n$). Across all datasets and settings, PGM-ATTACK causes a substantially larger increase than random poisoning.

			Poisoning			Upper Bound	
ε	Dataset	Original	PGM-attack	Random	Random-Adj.	Agnostic UB	Instance UB
16	Amzn	7.94M	8.98M (1.13×)	7.95M (1.00×)	8.06M (1.01×)	15.9M (2.01×)	14.5M (1.82×)
	Osmc	6.18M	6.76M (1.10×)	6.19M (1.00×)	6.25M (1.01×)	14.2M (2.30×)	11.2M (1.81×)
	Face	2.12M	2.33M (1.10×)	2.12M (1.00×)	2.14M (1.01×)	4.12M (1.94×)	3.64M (1.71×)
	YCSB	70.1K	176K (2.51×)	72.5K (1.03×)	74.1K (1.06×)	2.07M (29.5×)	352K (5.02×)
	Longitudes	164K	262K (1.59×)	166K (1.01×)	169K (1.03×)	2.16M (13.2×)	514K (3.13×)
	Longlat	452K	543K (1.20×)	456K (1.01×)	459K (1.01×)	2.45M (5.42×)	992K (2.19×)
	Uniform	203K	320K (1.57×)	209K (1.03×)	209K (1.03×)	2.2M (10.8×)	615K (3.02×)
	Normal	203K	320K (1.57×)	205K (1.01×)	209K (1.03×)	2.2M (10.8×)	614K (3.02×)
	Lognormal	203K	320K (1.57×)	205K (1.01×)	209K (1.03×)	2.2M (10.8×)	614K (3.02×)
32	Amzn	2.46M	2.97M (1.21×)	2.47M (1.00×)	2.52M (1.02×)	10.5M (4.25×)	5.15M (2.09×)
	Osmc	2.89M	3.14M (1.09×)	2.89M (1.00×)	2.92M (1.01×)	10.9M (3.77×)	5.31M (1.84×)
	Face	1.06M	1.16M (1.10×)	1.06M (1.00×)	1.07M (1.01×)	3.06M (2.90×)	1.82M (1.72×)
	YCSB	25.1K	63.9K (2.54×)	25.5K (1.02×)	25.9K (1.03×)	2.03M (80.6×)	113K (4.51×)
	Longitudes	62.6K	102K (1.63×)	62.9K (1.00×)	63.7K (1.02×)	2.06M (32.9×)	203K (3.25×)
	Longlat	219K	259K (1.18×)	221K (1.01×)	222K (1.01×)	2.22M (10.1×)	475K (2.17×)
	Uniform	53K	103K (1.94×)	54.5K (1.03×)	54.6K (1.03×)	2.05M (38.7×)	209K (3.95×)
	Normal	52.9K	103K (1.94×)	53.4K (1.01×)	54.5K (1.03×)	2.05M (38.8×)	209K (3.95×)
	Lognormal	52.9K	103K (1.94×)	53.5K (1.01×)	54.5K (1.03×)	2.05M (38.8×)	209K (3.95×)
64	Amzn	797K	962K (1.21×)	798K (1.00×)	810K (1.02×)	8.8M (11.0×)	1.73M (2.17×)
	Osmc	1.38M	1.49M (1.08×)	1.38M (1.00×)	1.39M (1.01×)	9.38M (6.82×)	2.55M (1.85×)
	Face	523K	577K (1.10×)	523K (1.00×)	528K (1.01×)	2.52M (4.82×)	905K (1.73×)
	YCSB	6.95K	25.6K (3.68×)	7.12K (1.02×)	7.36K (1.06×)	2.01M (289×)	46.6K (6.71×)
	Longitudes	27.8K	45.6K (1.64×)	27.9K (1.00×)	28.2K (1.01×)	2.03M (72.9×)	87K (3.13×)
	Longlat	112K	131K (1.17×)	113K (1.01×)	113K (1.01×)	2.11M (18.9×)	238K (2.12×)
	Uniform	13.5K	34.9K (2.60×)	13.9K (1.03×)	13.8K (1.03×)	2.01M (150×)	69K (5.12×)
	Normal	13.5K	34.9K (2.59×)	13.6K (1.01×)	13.9K (1.03×)	2.01M (149×)	69K (5.11×)
	Lognormal	13.5K	34.9K (2.59×)	13.6K (1.01×)	13.9K (1.03×)	2.01M (149×)	69K (5.11×)
128	Amzn	268K	335K (1.25×)	268K (1.00×)	273K (1.02×)	8.27M (30.9×)	619K (2.31×)
	Osmc	668K	723K (1.08×)	669K (1.00×)	674K (1.01×)	8.67M (13.0×)	1.23M (1.85×)
	Face	256K	283K (1.10×)	256K (1.00×)	259K (1.01×)	2.26M (8.80×)	444K (1.73×)
	YCSB	663	10.3K (15.5×)	759 (1.14×)	786 (1.19×)	2M (3018×)	17.6K (26.6×)
	Longitudes	13.7K	22.3K (1.63×)	13.8K (1.00×)	13.9K (1.01×)	2.01M (147×)	41.5K (3.02×)
	Longlat	58.1K	67.7K (1.16×)	58.9K (1.01×)	58.7K (1.01×)	2.06M (35.4×)	122K (2.11×)
	Uniform	3.42K	12.9K (3.78×)	3.53K (1.03×)	3.53K (1.03×)	2M (586×)	23.7K (6.91×)
	Normal	3.5K	13K (3.70×)	3.52K (1.01×)	3.6K (1.03×)	2M (572×)	23.7K (6.78×)
	Lognormal	3.49K	12.9K (3.71×)	3.52K (1.01×)	3.59K (1.03×)	2M (574×)	23.7K (6.81×)

all datasets and parameter settings, poison generation takes between 0.6 and 14.7 hours. Although this computational cost is non-negligible, it remains practical for an offline attacker because the poison keys can be generated in advance and subsequently inserted into the target index.

The generation time tends to increase as either ε or λ increases. The main reason is that these settings allow the selected poison keys to shorten the current segment more substantially. PGM-ATTACK processes the key sequence from left to right. At each step, it generates poison candidates for the segment starting at the current segment-start position, selects poison keys, and recomputes the resulting segmentation. If the selected poisons shorten the segment only slightly, the next segment starts relatively far to the right, and the algorithm soon moves to a mostly new region of the dataset. In contrast, when the shortening is large, the next segment starts much earlier than before poisoning. Consequently, the algorithm must again generate and evaluate poison candidates for a segment that substantially overlaps a previously processed region. This repeated recomputation over overlapping regions increases the total generation time. Larger ε permits greater reductions in segment length, while a larger poison budget λ increases the cumulative shortening caused by the selected poisons; both lead to more repeated processing.

A secondary reason for the increase with ε is that the segments before poisoning tend to contain more keys. For a segment containing c legitimate keys, DI-Consecutive requires $\mathcal{O}(c \log c)$ time to generate a poison candidate. Thus, as ε increases and the unpoisoned segment length c becomes larger, the cost of processing each candidate segment also grows superlinearly in c.

B. Additional Results on Index Performance

We further evaluate the practical impact of PGM-ATTACK on index size and query time. Because this extended evaluation covers more datasets, indexes, and poisoning methods than the evaluation in Section VI-E, we present the results as figures rather than tables for readability. Figures 19 and 20 report the index size and query time, respectively, for $\varepsilon = 128$ and 10% poisoning ($\lambda = 0.1n$). For each PGM-ATTACK bar,

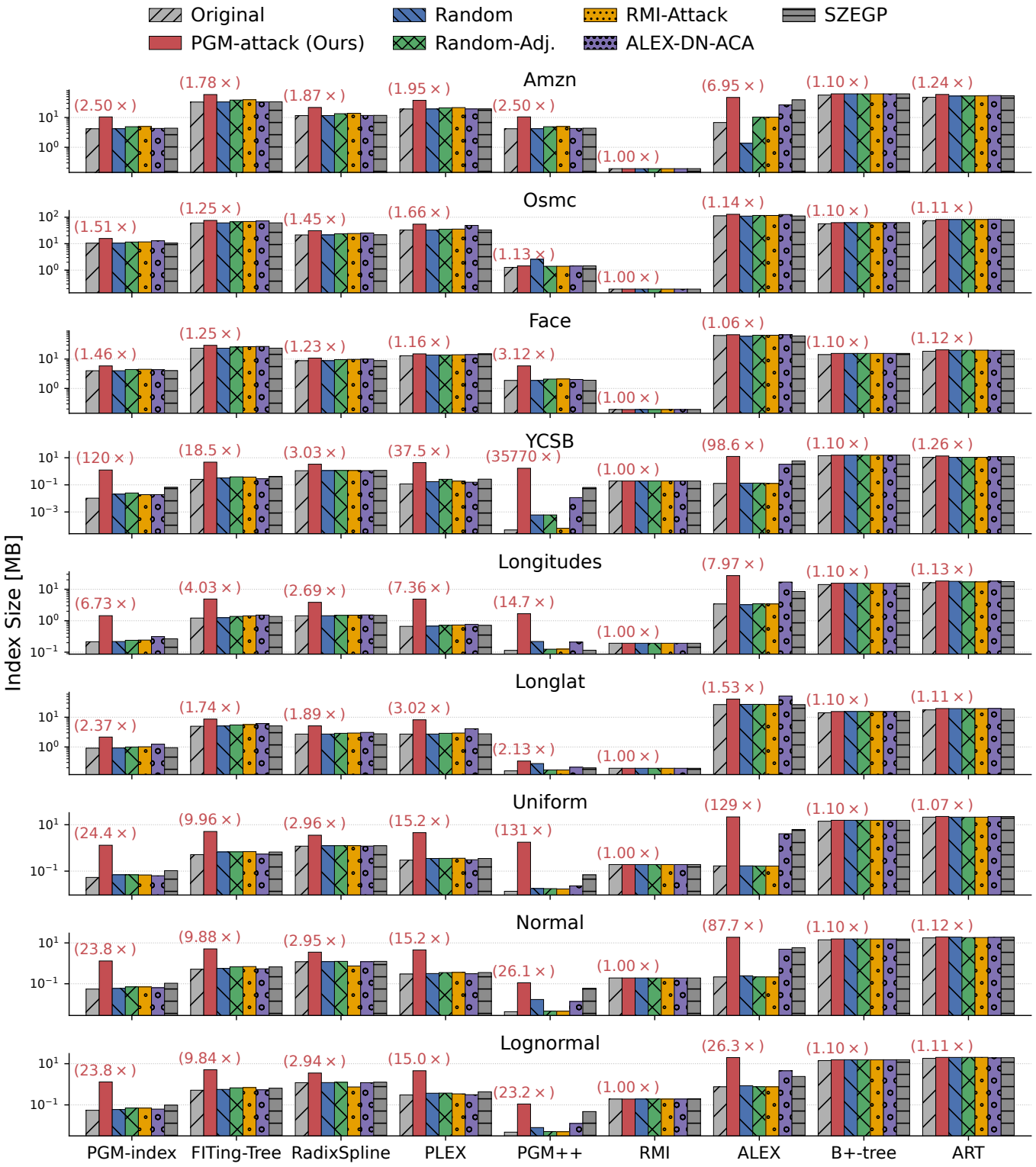


Fig. 19: Index sizes before and after 10% poisoning ($\lambda = 0.1n$) for $\varepsilon = 128$. Numbers above the PGM-ATTACK bars indicate ratios relative to the corresponding original indexes.

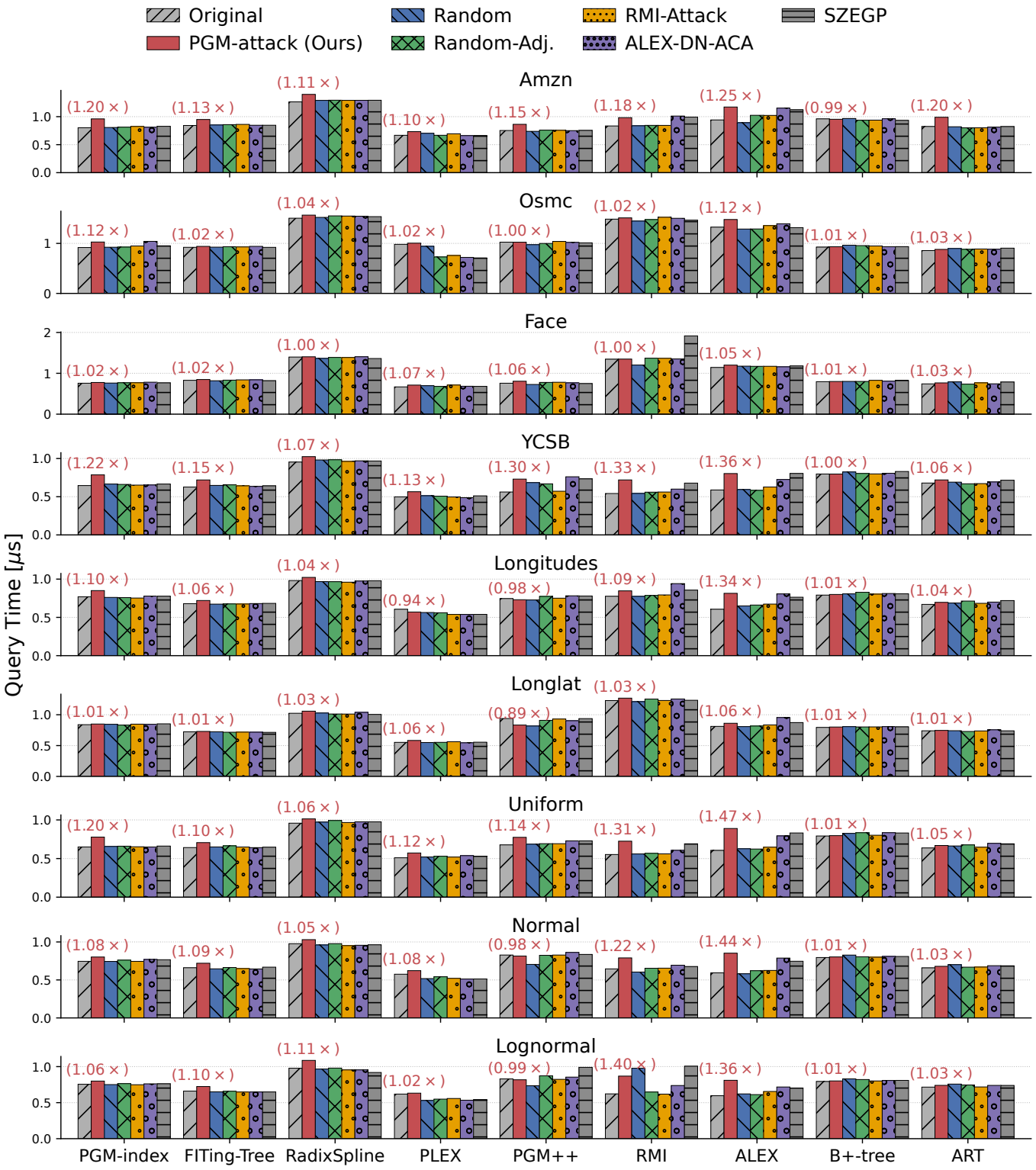


Fig. 20: Query times before and after 10% poisoning ($\lambda = 0.1n$) for $\varepsilon = 128$. Numbers above the PGM-ATTACK bars indicate ratios relative to the corresponding original indexes.

TABLE V: Results for m_{opt} after poisoning ($\lambda = 0.1n$). Across all datasets and settings, PGM-ATTACK causes a substantially larger increase than random poisoning.

			Poisoning			Upper Bound	
ε	Dataset	Original	PGM-attack	Random	Random-Adj.	Agnostic UB	Instance UB
16	Amzn	7.94M	12.1M (1.52×)	8.01M (1.01×)	9.12M (1.15×)	87.9M (11.1×)	21.2M (2.67×)
	Osmc	6.18M	9.18M (1.49×)	6.28M (1.02×)	6.92M (1.12×)	86.2M (14.0×)	17.4M (2.82×)
	Face	2.12M	3.08M (1.45×)	2.14M (1.01×)	2.34M (1.10×)	22.1M (10.4×)	5.36M (2.53×)
	YCSB	70.1K	810K (11.6×)	94.8K (1.35×)	111K (1.58×)	20.1M (286×)	1.3M (18.6×)
	Longitudes	164K	831K (5.05×)	180K (1.10×)	208K (1.26×)	20.2M (123×)	1.46M (8.86×)
	Longlat	452K	1.1M (2.44×)	473K (1.05×)	516K (1.14×)	20.5M (45.2×)	2.01M (4.45×)
	Uniform	203K	930K (4.57×)	262K (1.29×)	264K (1.30×)	20.2M (99.3×)	1.64M (8.04×)
	Normal	203K	929K (4.58×)	224K (1.10×)	264K (1.30×)	20.2M (99.5×)	1.64M (8.06×)
	Lognormal	203K	929K (4.58×)	224K (1.10×)	264K (1.30×)	20.2M (99.5×)	1.64M (8.06×)
32	Amzn	2.46M	4.79M (1.94×)	2.49M (1.01×)	3.03M (1.23×)	82.5M (33.5×)	9.19M (3.73×)
	Osmc	2.89M	4.3M (1.49×)	2.92M (1.01×)	3.22M (1.11×)	82.9M (28.7×)	8.26M (2.86×)
	Face	1.06M	1.53M (1.45×)	1.06M (1.01×)	1.16M (1.10×)	21.1M (20.0×)	2.69M (2.55×)
	YCSB	25.1K	361K (14.4×)	29.3K (1.17×)	32.8K (1.31×)	20M (797×)	563K (22.4×)
	Longitudes	62.6K	380K (6.07×)	66.1K (1.06×)	74.3K (1.19×)	20.1M (320×)	653K (10.4×)
	Longlat	219K	537K (2.45×)	226K (1.03×)	244K (1.11×)	20.2M (92.2×)	969K (4.42×)
	Uniform	53K	391K (7.38×)	68.4K (1.29×)	69.1K (1.30×)	20.1M (378×)	670K (12.6×)
	Normal	52.9K	391K (7.39×)	58.2K (1.10×)	68.8K (1.30×)	20.1M (379×)	670K (12.7×)
	Lognormal	52.9K	391K (7.39×)	58.2K (1.10×)	68.8K (1.30×)	20.1M (379×)	670K (12.7×)
64	Amzn	797K	1.73M (2.17×)	804K (1.01×)	929K (1.16×)	80.8M (101×)	3.24M (4.06×)
	Osmc	1.38M	2.06M (1.50×)	1.39M (1.01×)	1.52M (1.11×)	81.4M (59.2×)	3.97M (2.88×)
	Face	523K	760K (1.45×)	524K (1.00×)	575K (1.10×)	20.5M (39.2×)	1.35M (2.57×)
	YCSB	6.95K	169K (24.2×)	8.73K (1.26×)	10.3K (1.49×)	20M (2877×)	272K (39.1×)
	Longitudes	27.8K	185K (6.66×)	28.7K (1.03×)	31.7K (1.14×)	20M (720×)	312K (11.2×)
	Longlat	112K	271K (2.42×)	114K (1.02×)	123K (1.10×)	20.1M (180×)	483K (4.32×)
	Uniform	13.5K	177K (13.2×)	17.5K (1.30×)	17.6K (1.30×)	20M (1487×)	294K (21.8×)
	Normal	13.5K	177K (13.1×)	14.8K (1.10×)	17.5K (1.30×)	20M (1484×)	294K (21.8×)
	Lognormal	13.5K	177K (13.1×)	14.8K (1.10×)	17.6K (1.30×)	20M (1483×)	294K (21.8×)
128	Amzn	268K	670K (2.50×)	269K (1.01×)	315K (1.17×)	80.3M (300×)	1.23M (4.60×)
	Osmc	668K	1M (1.51×)	671K (1.01×)	738K (1.10×)	80.7M (121×)	1.93M (2.88×)
	Face	256K	374K (1.46×)	257K (1.00×)	282K (1.10×)	20.3M (79.0×)	668K (2.60×)
	YCSB	663	79.8K (120×)	1.32K (1.99×)	1.6K (2.42×)	20M (30167×)	130K (196×)
	Longitudes	13.7K	92.4K (6.73×)	13.9K (1.01×)	15.3K (1.11×)	20M (1458×)	154K (11.2×)
	Longlat	58.1K	138K (2.37×)	59.1K (1.02×)	63.7K (1.10×)	20.1M (345×)	245K (4.22×)
	Uniform	3.42K	83.7K (24.5×)	4.45K (1.30×)	4.44K (1.30×)	20M (5847×)	136K (39.8×)
	Normal	3.5K	83.5K (23.8×)	3.77K (1.08×)	4.49K (1.28×)	20M (5707×)	136K (38.9×)
	Lognormal	3.49K	83.4K (23.9×)	3.81K (1.09×)	4.51K (1.29×)	20M (5735×)	136K (39.1×)

we additionally display the ratio relative to the corresponding original, unpoisoned index above the bar.

Poisoning Methods. In addition to our proposed PGM-ATTACK, we evaluate two random baselines, RANDOM and RANDOM-ADJ. RANDOM selects poison keys uniformly at random, without replacement, from the unoccupied integers within the legitimate key range. Motivated by Theorem 1, RANDOM-ADJ. instead samples uniformly from unoccupied integers adjacent to legitimate keys. We also evaluate the poisoning attack proposed for RMIs [5], which we refer to as RMI-ATTACK. Although RMI-ATTACK was specifically designed to maximize the squared prediction error of RMI models, we apply the poison keys generated by every method to every evaluated index to examine their transferability across index architectures.

We further evaluate two methods proposed as algorithmic complexity attacks (ACAs) against ALEX. The first is a space ACA targeting ALEX data nodes, which we denote by ALEX-DN-ACA [8]. The second, denoted by SZEGP, is a space ACA that induces repeated structural expansion in ALEX through adversarial insertions [9]. Both methods were originally designed to increase the memory consumption of an already constructed ALEX by dynamically inserting adversarially selected keys. This differs from our poisoning setting, in which the attacker injects poison keys into the dataset before index construction. To ensure a consistent comparison, we evaluate all methods under our poisoning setting: each index is bulk-loaded from the dataset containing the generated poison keys, rather than being attacked through online insertions after construction.

Effect on PLA-Based Learned Indexes. PGM-ATTACK is not only effective against PGM-index, but also effective against all four other PLA-based learned indexes. The largest multiplicative increases tend to occur when the original index is particularly small, that is, when the index benefits strongly from the regularity and linear approximability of the original key distribution. In such cases, poisoning removes much of this advantage by creating many local regions that cannot be accurately represented using a small number of linear segments.

TABLE VI: m_{opt} after PGM-ATTACK when the PGM-index is constructed with $\varepsilon_{\text{true}}$, but poisons are generated assuming that the PGM-index parameter is $\varepsilon_{\text{attack}}$. $\lambda = 0.1n$. Row labels give $\varepsilon_{\text{attack}}$.

Amzn	$\varepsilon_{\text{true}} = 16$	$\varepsilon_{\text{true}} = 32$	$\varepsilon_{\text{true}} = 64$	$\varepsilon_{\text{true}} = 128$
16	**12.1M (1.52×)**	3.19M (1.30×)	1.04M (1.31×)	386K (1.44×)
32	10.2M (1.28×)	**4.79M (1.94×)**	1.05M (1.32×)	365K (1.36×)
64	9.2M (1.16×)	3.55M (1.44×)	**1.73M (2.17×)**	395K (1.47×)
128	8.52M (1.07×)	3.03M (1.23×)	1.33M (1.67×)	**670K (2.50×)**
Osmc	$\varepsilon_{\text{true}} = 16$	$\varepsilon_{\text{true}} = 32$	$\varepsilon_{\text{true}} = 64$	$\varepsilon_{\text{true}} = 128$
16	**9.18M (1.49×)**	3.47M (1.20×)	1.6M (1.17×)	780K (1.17×)
32	7.48M (1.21×)	**4.3M (1.49×)**	1.66M (1.21×)	773K (1.16×)
64	6.84M (1.11×)	3.53M (1.22×)	**2.06M (1.50×)**	809K (1.21×)
128	6.53M (1.06×)	3.22M (1.11×)	1.69M (1.23×)	**1M (1.51×)**
Face	$\varepsilon_{\text{true}} = 16$	$\varepsilon_{\text{true}} = 32$	$\varepsilon_{\text{true}} = 64$	$\varepsilon_{\text{true}} = 128$
16	**3.08M (1.45×)**	1.19M (1.13×)	581K (1.11×)	286K (1.11×)
32	2.59M (1.22×)	**1.53M (1.45×)**	594K (1.14×)	286K (1.12×)
64	2.4M (1.13×)	1.3M (1.23×)	**760K (1.45×)**	293K (1.15×)
128	2.29M (1.08×)	1.2M (1.13×)	646K (1.23×)	**374K (1.46×)**
YCSB	$\varepsilon_{\text{true}} = 16$	$\varepsilon_{\text{true}} = 32$	$\varepsilon_{\text{true}} = 64$	$\varepsilon_{\text{true}} = 128$
16	**810K (11.6×)**	144K (5.74×)	47.6K (6.85×)	15.1K (22.7×)
32	622K (8.87×)	**361K (14.4×)**	64.7K (9.30×)	24.1K (36.3×)
64	328K (4.67×)	264K (10.5×)	**169K (24.2×)**	35.6K (53.7×)
128	206K (2.93×)	162K (6.44×)	103K (14.8×)	**79.8K (120×)**
Longitudes	$\varepsilon_{\text{true}} = 16$	$\varepsilon_{\text{true}} = 32$	$\varepsilon_{\text{true}} = 64$	$\varepsilon_{\text{true}} = 128$
16	**831K (5.05×)**	174K (2.77×)	66.7K (2.40×)	33.1K (2.41×)
32	490K (2.98×)	**380K (6.07×)**	79.7K (2.86×)	29.5K (2.15×)
64	324K (1.97×)	220K (3.51×)	**185K (6.66×)**	39.6K (2.89×)
128	245K (1.49×)	142K (2.26×)	106K (3.82×)	**92.4K (6.73×)**
Longlat	$\varepsilon_{\text{true}} = 16$	$\varepsilon_{\text{true}} = 32$	$\varepsilon_{\text{true}} = 64$	$\varepsilon_{\text{true}} = 128$
16	**1.1M (2.44×)**	323K (1.47×)	148K (1.32×)	76.1K (1.31×)
32	765K (1.69×)	**537K (2.45×)**	164K (1.46×)	74.1K (1.27×)
64	611K (1.35×)	375K (1.71×)	**271K (2.42×)**	84.2K (1.45×)
128	534K (1.18×)	298K (1.36×)	190K (1.70×)	**138K (2.37×)**
Uniform	$\varepsilon_{\text{true}} = 16$	$\varepsilon_{\text{true}} = 32$	$\varepsilon_{\text{true}} = 64$	$\varepsilon_{\text{true}} = 128$
16	**930K (4.57×)**	193K (3.63×)	69K (5.12×)	28.1K (8.21×)
32	588K (2.89×)	**391K (7.38×)**	79.8K (5.93×)	27.3K (7.99×)
64	403K (1.98×)	243K (4.59×)	**177K (13.2×)**	36.7K (10.7×)
128	303K (1.49×)	154K (2.91×)	111K (8.23×)	**83.7K (24.5×)**
Normal	$\varepsilon_{\text{true}} = 16$	$\varepsilon_{\text{true}} = 32$	$\varepsilon_{\text{true}} = 64$	$\varepsilon_{\text{true}} = 128$
16	**929K (4.58×)**	192K (3.64×)	69K (5.12×)	28.3K (8.06×)
32	587K (2.89×)	**391K (7.39×)**	79.7K (5.91×)	27.2K (7.76×)
64	403K (1.98×)	243K (4.60×)	**177K (13.1×)**	36.7K (10.5×)
128	302K (1.49×)	153K (2.90×)	110K (8.18×)	**83.5K (23.8×)**
Lognormal	$\varepsilon_{\text{true}} = 16$	$\varepsilon_{\text{true}} = 32$	$\varepsilon_{\text{true}} = 64$	$\varepsilon_{\text{true}} = 128$
16	**929K (4.58×)**	192K (3.63×)	69K (5.12×)	28.3K (8.10×)
32	587K (2.89×)	**391K (7.39×)**	79.7K (5.91×)	27.2K (7.81×)
64	403K (1.98×)	243K (4.60×)	**177K (13.1×)**	36.7K (10.5×)
128	302K (1.49×)	153K (2.90×)	110K (8.16×)	**83.4K (23.9×)**

This effect is most pronounced on YCSB, although PGM-ATTACK produces greater performance degradation than the other poisoning methods in nearly all cases on the remaining real-world and synthetic datasets. On YCSB, relative to the original index, PGM-ATTACK increases the index size by up to $18.5\times$ for FITing-Tree, $3.03\times$ for RadixSpline, $37.5\times$ for PLEX, and $35{,}770\times$ for PGM++. The exceptionally large ratio for PGM++ partly reflects its extremely small original index size on YCSB. Across all other real-world and synthetic datasets, PGM-ATTACK increases the sizes of all PLA-based learned indexes by factors ranging from $1.13\times$ to $130\times$. In nearly every case, it produces a larger increase than any competing poisoning method.

These results suggest that PGM-ATTACK exploits the underlying PLA approximation problem rather than an implementation-specific property of the PGM-index. Its poison keys therefore transfer to learned indexes that employ different PLA construction algorithms, recursive organizations, or auxiliary search structures.

The increase in query time is less pronounced than the increase in index size. Nevertheless, in most cases, PGM-ATTACK causes the largest query-time degradation among the evaluated poisoning methods, increasing query time by up to $1.30\times$. This slowdown is likely attributable, at least in part, to the larger structures induced by the increased number of segments, which may lead to additional cache misses and traversal overhead.

Effect on Other Learned Indexes. PGM-ATTACK can also substantially increase both the index size and query time of ALEX. In particular, it increases the ALEX index size by $98.6\times$ on YCSB and by $129\times$ on Uniform. These large increases can be explained by the fact that ALEX's bulk-loading algorithm adaptively determines its index structure based on model accuracy. During construction, ALEX evaluates the model accuracy at each node and decides whether to retain the keys in a single data node or partition them among multiple child nodes. Lower model accuracy can therefore lead to a more highly partitioned structure and a larger overall index.

Because ALEX uses linear models to navigate its nodes, the poison keys generated by PGM-ATTACK likely also increase the prediction errors of these models. These results demonstrate that the transferability of PGM-ATTACK extends beyond the family of PLA-based learned indexes.

Comparison with RMI-ATTACK. Notably, PGM-ATTACK is at least as effective as RMI-ATTACK, even on RMI, for which the latter was specifically designed. On RMI, PGM-ATTACK increases query time by up to $1.40\times$, whereas RMI-ATTACK increases it by at most $1.04\times$.

One possible explanation is the difference between the objectives optimized by the two attacks. RMI-ATTACK selects poison keys to maximize the squared prediction error of the RMI models. However, an RMI answers a query by performing a last-mile search, such as linear or exponential search, around the predicted position. Squared error may therefore be imperfectly aligned with the actual query-processing cost: a small number of very large errors can dominate the objective without proportionally increasing the average last-mile search cost. In contrast, PGM-ATTACK targets local worst-case approximation error and places poison keys in regions where they substantially disrupt linear approximation. This objective may be more closely aligned with the last-mile search cost incurred by learned indexes.

The optimization procedure used by RMI-ATTACK may also limit its effectiveness. In particular, it assumes that leaf models are responsible for equal numbers of keys and employs a hill-climbing procedure initialized from a uniform poison allocation. Consequently, the resulting poison set may remain

TABLE VII: Poison generation time (hours).

	$\lambda = 0.01\,n$				$\lambda = 0.1\,n$			
Dataset	$\varepsilon = 16$	$\varepsilon = 32$	$\varepsilon = 64$	$\varepsilon = 128$	$\varepsilon = 16$	$\varepsilon = 32$	$\varepsilon = 64$	$\varepsilon = 128$
Amzn	3.1	2.9	4.3	4.7	4.9	5.8	8.7	12.3
Osmc	2.9	2.5	2.6	2.3	4.9	4.8	4.8	5.0
Face	0.9	0.8	0.8	0.6	1.3	1.2	1.0	1.0
YCSB	1.5	1.2	2.0	3.3	3.5	4.2	6.0	14.7
Longitudes	1.0	0.9	1.1	1.1	3.0	4.3	5.0	5.3
Longlat	0.7	0.7	0.7	0.6	2.2	2.7	2.7	2.6
Uniform	1.0	1.2	1.6	1.9	2.3	3.7	5.3	7.3
Normal	1.0	1.2	1.6	1.8	2.3	3.9	5.5	7.1
Lognormal	1.0	1.2	1.6	1.8	2.3	3.6	5.3	7.2

far from the optimum for a particular key distribution or RMI configuration.

Comparison with Attacks on ALEX. A similar pattern arises for ALEX. Under our evaluation setting, PGM-ATTACK causes greater degradation than the attacks proposed specifically for ALEX [8], [9]. This difference is largely attributable to the threat models for which these ACA methods were designed. They exploit algorithmic behavior triggered by online insertions into an already constructed ALEX, such as inefficient resizing, splitting, or structural expansion, rather than constructing a poisoned dataset from which ALEX is subsequently bulk-loaded.

When the keys generated by these attacks are instead included during bulk loading, ALEX can jointly optimize its initial structure over both legitimate and adversarial keys, avoiding many of the transient inefficiencies that arise during online updates. Bulk loading is therefore generally a more favorable setting for the index than adversarial online insertion. The effectiveness of PGM-ATTACK even in this setting suggests that it does not merely exploit an inefficient update procedure. Rather, it reveals a broader vulnerability in the model-based construction objective itself: carefully placed keys can make the data intrinsically difficult to represent using the simple local models on which the index relies.

Transferability across Learned Indexes. Finally, PGM-ATTACK exhibits substantially greater transferability than the prior index-specific attacks. The effects of RMI-ATTACK, ALEX-DN-ACA, and SZEGP are generally limited outside the architectures for which they were designed. By contrast, PGM-ATTACK consistently degrades a broad range of learned indexes.

We attribute this transferability to a design principle shared by many learned indexes: they partition the key space into regions and represent each region using a small, simple model, often a linear model. PGM-ATTACK concentrates poison keys at locations where they substantially increase local approximation difficulty and worst-case prediction error. Because these quantities are closely related to the number of required models, the complexity of the index structure, and the cost of last-mile search, the resulting poison keys can remain effective across substantially different learned-index architectures.

Algorithm 3 PLA construction in PGM-index with duplicate keys

Require:
Nondecreasing keys array $\mathcal{X} = (x_1, x_2, \ldots, x_n)$;
Maximum error tolerance ε.
Ensure: A PLA built for $\mathcal{X}$ with error at most ε.

```
PLA ← initializePLA(ε)
PLA.addPoint(x_1, 1)
for i ← 2 to n do
    if x_i = x_{i−1} then
        if i < n ∧ x_i + 1 < x_{i+1} then
            PLA.addPoint(x_i + 1, i)
    else
        PLA.addPoint(x_i, i)
return PLA
```

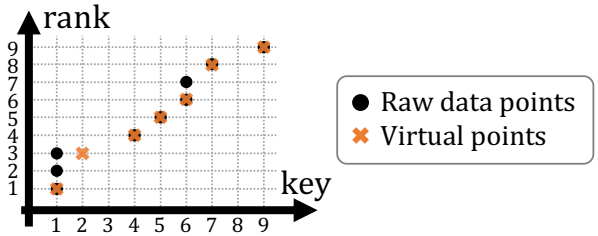


Fig. 21: PLA construction with duplicate keys in the PGM-index. Instead of directly constructing the PLA from the original key-rank pairs, the PGM-index constructs the PLA over a set of virtual points introduced to handle duplicates.

APPENDIX E
DUPLICATE-ALLOWED POISONING

In this section, we discuss poisoning in the duplicate-allowed setting. We first describe how duplicates are handled in the actual PGM-index implementation. We then discuss how to generate poisons under this setting. Finally, we experimentally evaluate PGM-ATTACK in this setting.

A. PGM-index with Duplicates

When given a sorted integer array containing duplicate keys, the PGM-index does not construct the PLA directly from the original key-rank pairs. Instead, it constructs the PLA over virtual points introduced to handle duplicates. Specifically, it

first adds $(x_1, 1)$. Then, for each position $i \in \{2, 3, \ldots, n\}$, if x_i differs from the previous key, it adds (x_i, i). If x_i is a duplicate, it does not add that point directly; instead, only when x_i is at the end of a duplicate run and there is a gap before the next distinct key, it adds $(x_i + 1, i)$. Algorithm 3 presents the pseudocode, and Figure 21 illustrates the resulting virtual points. The PLA is then constructed with error at most ε on these virtual points.

This keeps the construction feasible even when a key occurs more than $2\varepsilon + 1$ times; if the raw data points are used directly, PLA construction becomes infeasible in this case (see Figure 21). At the same time, the resulting PLA can still answer lower-bound and related queries correctly. In particular, $(x_i + 1, i)$ captures the information immediately after the duplicate run of value x_i. As a result, even with duplicate keys, the PGM-index can properly construct the approximate position and correction range used internally, allowing its final search procedure to return the correct result for lower-bound-style queries.

B. Poison Generation with Duplicates

At a high level, our algorithm for generating poisons in the duplicate-allowed setting is the same as the algorithm for the duplicate-forbidden setting (Algorithm 1). The only difference is the function DI-CONSECUTIVE($\mathcal{K}_{\text{seg}}, \lambda_{\text{cand}}$) in Algorithm 1, namely, the function that generates λ poisons for a given segment $\mathcal{K}_{\text{seg}}$.

In the duplicate-forbidden setting, we used the discrete-intercept algorithm to efficiently evaluate candidate poison sets consisting of consecutive integers. In the duplicate-allowed setting, we again use the discrete-intercept algorithm, but now we evaluate poison sets consisting of a single integer. This design is motivated by Theorem 2, which shows that, in the duplicate-allowed maximum-error maximization problem, an optimal poison set can consist of a single legitimate key. Motivated by this result, we efficiently evaluate such singleton candidates using the discrete-intercept algorithm and select the poison set that minimizes the number of covered legitimate keys.

Note that, when selecting this single integer and evaluating the poison set, we use *virtual points* rather than raw data points. Specifically, for each key appearing among the virtual points introduced in Algorithm 3, we form a poison set consisting of λ copies of that key and evaluate it. As in Appendix B, each candidate can be evaluated in $\mathcal{O}(|\mathcal{I}| \log c)$ time, where c is the number of legitimate keys currently coverable by one segment. Since the number of candidates is $\mathcal{O}(c)$, the total time complexity is $\mathcal{O}(|\mathcal{I}| c \log c)$. By using virtual points in this way, the poison-generation algorithm partially reflects the behavior of the PGM-index PLA construction algorithm.

C. Experimental Evaluation

Here, we evaluate how much PGM-ATTACK can increase the number of segments when duplicates are allowed.

Datasets. To evaluate the effect of PGM-ATTACK in the duplicate-allowed setting, we use three real-world datasets containing duplicates and one synthetic dataset with duplicates:

- From SOSD [24], we use the **Wiki** dataset. It contains 200.0M keys in total, of which 90.0M are unique.
- Following the PGM-index paper [2] and subsequent work, we use the **Weblogs** and **IoT** datasets. However, the MgBench dataset [42] used in that line of work is no longer publicly available. We therefore use the mgbench dataset released by Andrew Crotty [43]. In this benchmark, bench2 corresponds to **Weblogs** and bench3 to **IoT**. They contain 75.7M and 109.0M keys, respectively, and the numbers of unique keys are 18.7M and 108.9M.
- As a synthetic dataset, we generate a **Zipf** dataset. It contains 200.0M keys in total, of which 34.8M are unique.

Methods. We compare PGM-ATTACK with RANDOM and RANDOM-ADJ. Note, however, that these random baselines differ slightly from those in the duplicate-forbidden setting, although the underlying policy is analogous.

For RANDOM, in the duplicate-forbidden setting we sampled uniformly from the allowed integers, i.e., integers in the key range $[k_1, k_n]$ that do not belong to the legitimate keys. In the duplicate-allowed setting, by contrast, every integer in the key range is allowed, so we simply sample λ integers uniformly at random from that key range.

For RANDOM-ADJ., in the duplicate-forbidden setting we sampled uniformly from integers adjacent to legitimate keys. In the duplicate-allowed setting, since integers already contained in the legitimate keys are also allowed, we instead sample λ integers uniformly at random, with replacement, from the legitimate keys.

Results. Table VIII reports how much PGM-ATTACK increases the number of segments in the duplicate-allowed setting.

For every dataset, PGM-ATTACK increases m_{opt} relative to the legitimate-key baseline: by up to $4.68\times$ at 1% poisoning ($\lambda = 0.01\,n$) and up to $36.1\times$ at 10% poisoning ($\lambda = 0.1\,n$). In contrast, RANDOM and RANDOM-ADJ. barely change m_{opt}, by at most $1.24\times$ and $1.30\times$, respectively. These results show that, even when duplicates are allowed, PGM-ATTACK yields larger increases in m_{opt} than random insertions.

We also observe, as in the duplicate-forbidden setting, that the amplification factor of PGM-ATTACK tends to grow with ε. For example, when $\lambda = 0.1\,n$, the amplification factor is at most $8.01\times$ for $\varepsilon = 16$, but rises to $36.1\times$ for $\varepsilon = 128$. As discussed in Section VI-D, this trend can be understood intuitively in the same way as in the duplicate-forbidden setting.

TABLE VIII: Results for m_{opt} after duplicate-allowed poisoning. PGM-ATTACK increases m_{opt} relative to legitimate keys (by up to $4.68\times$ at $\lambda = 0.01\,n$ and up to $36.1\times$ at $\lambda = 0.1\,n$), whereas RANDOM and RANDOM-ADJ. barely change m_{opt} (by at most $1.24\times$ and $1.30\times$, respectively). The amplification factor for PGM-ATTACK tends to grow with ε.

			$\lambda = 0.01\,n$			$\lambda = 0.1\,n$		
ε	Dataset	Original	PGM-ATTACK	RANDOM	RANDOM-ADJ.	PGM-ATTACK	RANDOM	RANDOM-ADJ.
16	Weblogs	291K	340K (1.17×)	292K (1.00×)	295K (1.01×)	583K (2.00×)	297K (1.02×)	331K (1.14×)
	IoT	178K	225K (1.27×)	179K (1.01×)	181K (1.02×)	526K (2.96×)	186K (1.05×)	206K (1.16×)
	Wiki	228K	334K (1.46×)	230K (1.01×)	233K (1.02×)	937K (4.11×)	245K (1.07×)	280K (1.23×)
	Zipf	83.4K	151K (1.81×)	85.4K (1.02×)	85.5K (1.03×)	668K (8.01×)	104K (1.24×)	105K (1.26×)
32	Weblogs	122K	145K (1.19×)	123K (1.00×)	124K (1.01×)	267K (2.18×)	124K (1.01×)	139K (1.14×)
	IoT	78.3K	101K (1.29×)	78.5K (1.00×)	79.3K (1.01×)	253K (3.23×)	80.6K (1.03×)	89.6K (1.14×)
	Wiki	85.3K	130K (1.53×)	85.8K (1.01×)	86.8K (1.02×)	414K (4.86×)	89.4K (1.05×)	101K (1.18×)
	Zipf	24.9K	58.1K (2.33×)	25.5K (1.02×)	25.6K (1.03×)	326K (13.1×)	30.2K (1.21×)	31.8K (1.28×)
64	Weblogs	49.3K	59.8K (1.21×)	49.4K (1.00×)	50.1K (1.01×)	119K (2.41×)	49.8K (1.01×)	56.2K (1.14×)
	IoT	36.3K	47.2K (1.30×)	36.4K (1.00×)	36.8K (1.01×)	123K (3.39×)	36.9K (1.02×)	41K (1.13×)
	Wiki	37.5K	56.9K (1.52×)	37.7K (1.01×)	38K (1.01×)	194K (5.16×)	38.7K (1.03×)	42.4K (1.13×)
	Zipf	7.24K	23.7K (3.27×)	7.34K (1.01×)	7.44K (1.03×)	160K (22.1×)	8.58K (1.19×)	9.32K (1.29×)
128	Weblogs	20K	25K (1.25×)	20K (1.00×)	20.3K (1.01×)	53.4K (2.67×)	20.2K (1.01×)	22.7K (1.14×)
	IoT	16.3K	22.2K (1.37×)	16.3K (1.00×)	16.5K (1.02×)	60.4K (3.72×)	16.5K (1.01×)	18.5K (1.14×)
	Wiki	19.2K	28K (1.46×)	19.3K (1.01×)	19.4K (1.01×)	94.7K (4.93×)	19.5K (1.02×)	21.1K (1.10×)
	Zipf	2.18K	10.2K (4.68×)	2.2K (1.01×)	2.25K (1.03×)	78.7K (36.1×)	2.52K (1.16×)	2.83K (1.30×)